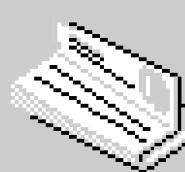
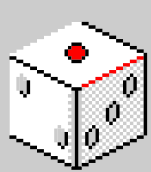
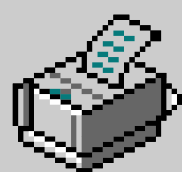
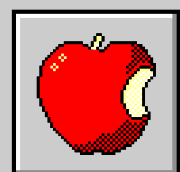
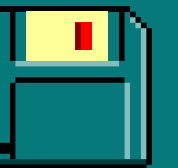
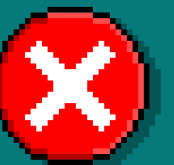
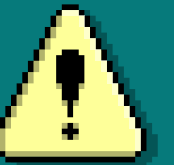
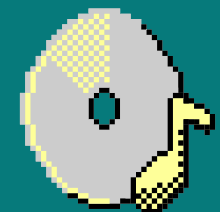
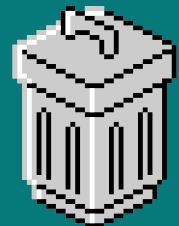




Sleep Stage Classification Using Deep Learning

**Course Instructor:
Dr. Ibrahim Sadek**



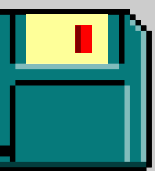
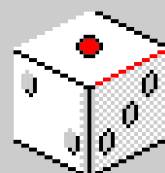
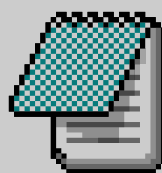
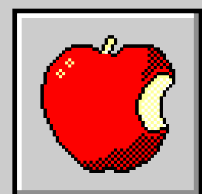
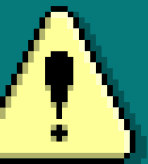
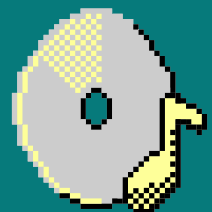
11:11P
M



Team members :

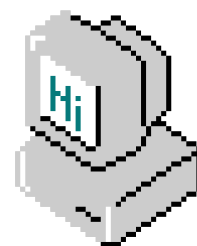
1.Eman Sabry

2. Sara Mohammed

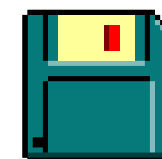


Agenda

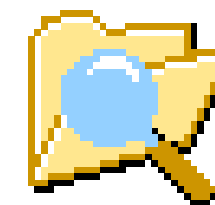
Topics Covered



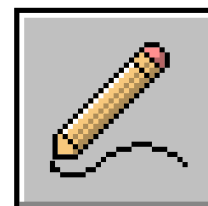
Problem



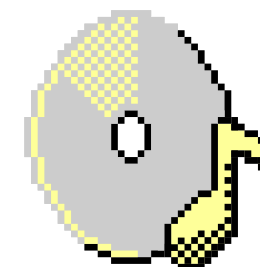
git repo



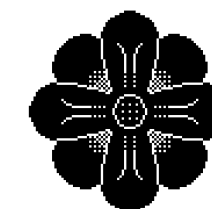
Dataset



Visualization



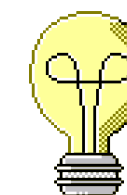
Data split



preprocessing

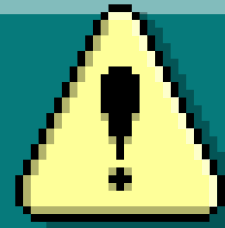


Model



Test

Start

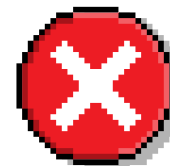


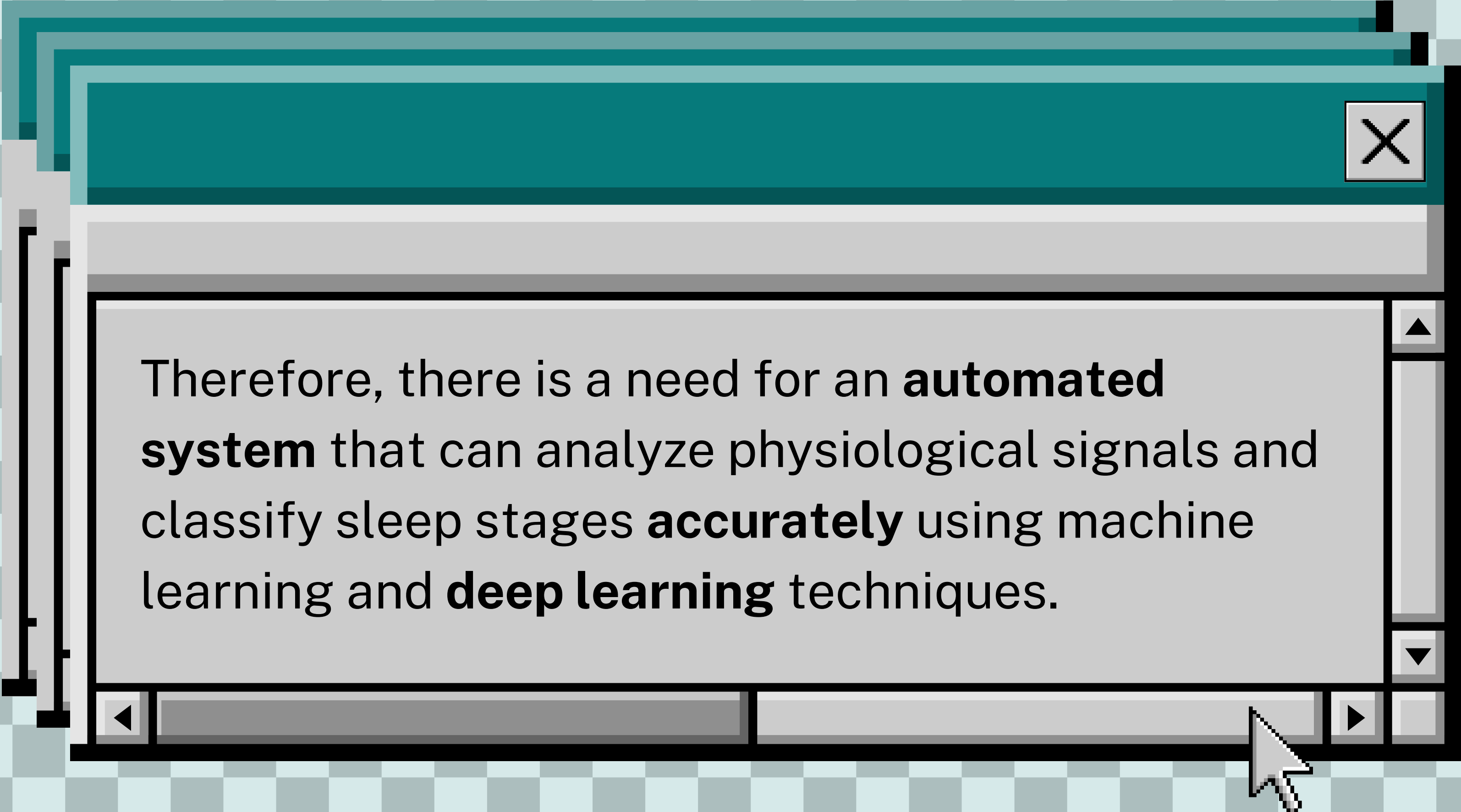
Problem

Sleep disorders affect millions of people worldwide and require accurate monitoring of sleep stages.

Traditional sleep analysis is performed manually by sleep experts using polysomnography (PSG) signals, which is:

- **Time-consuming**
- **Expensive**
- **Prone to human error**





Therefore, there is a need for an **automated system** that can analyze physiological signals and classify sleep stages **accurately** using machine learning and **deep learning** techniques.

Data description

What is the MESA Sleep Dataset?

The MESA Sleep Dataset is a public medical dataset containing overnight sleep recordings from real patients.

It includes:

- Physiological sleep signals (PSG)
- Expert-labeled sleep stages
- Multiple channels such as EEG, EOG, and EMG

It is widely used for sleep stage classification and sleep analysis research.

In this project, we used the EOG-R channel to classify sleep stages using deep learning.

sleepdata.org

GitHub Repository

The project code and implementation are available in the GitHub repository.

✕

🔍

📁 signals-project- Public

📁 main ▾

📁 2 Branches

📁 0 Tags

🔍 Go to file

ⓧ Add file ▾

📄 Code ▾

📁 eman-sabry Resolve merge conflict

5cd2bcb · 1 hour ago

🕒 15 Commits

📄 .gitignore	change learnig rate	1 hour ago
📄 README.md	change learnig rate	1 hour ago
📄 emo.ipynb	eman model	3 weeks ago
📄 finalModel.ipynb	Resolve merge conflict	1 hour ago
📄 requirements.txt	eman model	3 weeks ago
📄 sara_2_el_best - Copy.ipynb	add best model	2 hours ago

Contributors 2

👤 eman-sabry

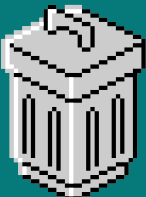
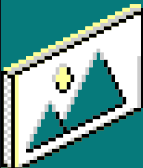
👤 saramohamedhamza Sara Mohmm...

Languages

● Jupyter Notebook 100.0%

link

🖱



Dataset

The dataset contains physiological sleep recordings collected during overnight sleep studies.

Data File	Extension	Full Meaning	Tool Used	File Contents
Signals	.edf	European Data Format	MNE	Contains raw physiological recordings for each subject, including biomedical signals such as EEG (Electroencephalography), EOG (Electrooculography), EMG (Electromyography), ECG (Electrocardiography), respiratory airflow, oxygen saturation, and other sleep monitoring channels.
Annotations	.xml	Extensible Markup Language	Python xml.etree.ElementTree	Contains expert-labeled sleep stage annotations and event metadata, including sleep stage transitions, start time, duration.

Each file is named by patient ID
, for example: (mesa-sleep-0001)

Sleep Stage Classes



The diagram shows three rectangular boxes representing sleep stages. Each box has a teal header bar and a white body. The first box is labeled 'Wake', the second 'NREM Sleep', and the third 'REM Sleep'. A yellow arrow points from the 'NREM Sleep' box to a pink callout box below it. A mouse cursor is visible near the top right of the 'REM Sleep' box.

Wake

NREM
Sleep

REM
Sleep

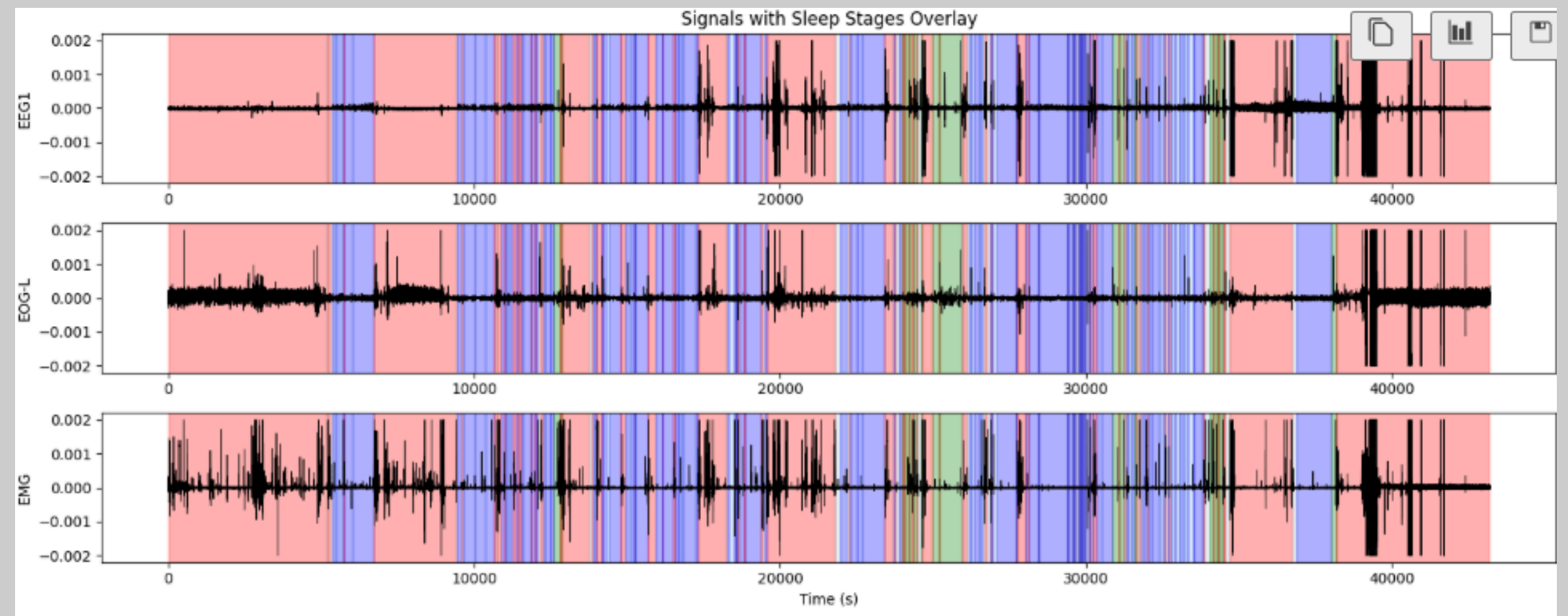
Non-Rapid Eye Movement sleep consists of:

- N1 → Light sleep
- N2 → Deeper sleep
- N3 → Deep sleep (slow-wave sleep)

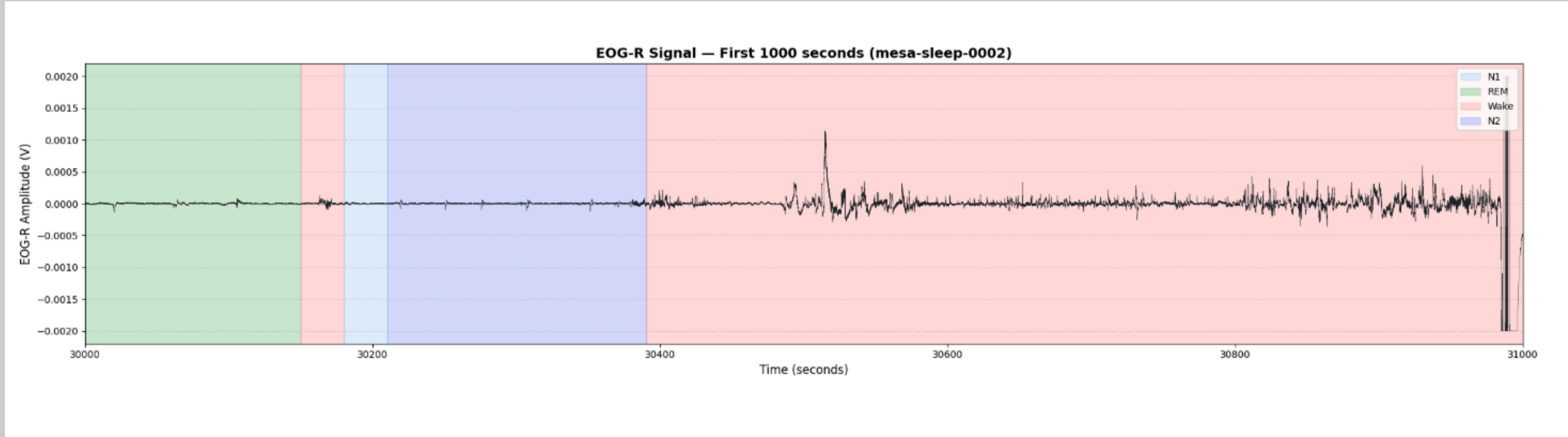
total calcification class is 5

Visualization of Signals with Sleep Stages

Wake	Red
Stage 1	Light
Stage 2	Blue
Stage 3	Dark Blue
REM	Green



Visualization EOG-R



Wake	Red
Stage 1	Light Blue
Stage 2	Blue
Stage 3	Dark Blue
REM	Green

what's inside ?

```
Creating raw.info structure...
```

```
Reading 0 ... 11058943 = 0.000 ... 43198.996 secs...
```

```
<Info | 8 non-empty values
```

```
bads: []
```

```
ch_names: EKG, EOG-L, EOG-R, EMG, EEG1, EEG2, EEG3, Pres, Flow, Snore, ...
```

```
chs: 27 EEG
```

```
custom_ref_applied: False
```

```
highpass: 0.0 Hz
```

```
lowpass: 128.0 Hz
```

```
meas_date: 1985-01-01 20:29:59 UTC
```

```
nchan: 27
```

```
projs: []
```

```
sfreq: 256.0 Hz
```

```
subject_info: <subject_info | his_id: >
```

```
>
```

```
['EKG', 'EOG-L', 'EOG-R', 'EMG', 'EEG1', 'EEG2', 'EEG3', 'Pres', 'Flow', 'Snore', 'Thor',  
43198.99609375]
```

```
import mne
```

```
# Load the signal
```

```
raw = mne.io.read_raw_edf('Dataset/Signals/mesa-sleep-0001.edf', preload=True)
```

```
# See what's inside
```

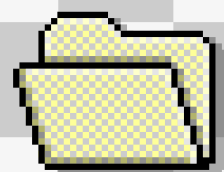
```
print(raw.info) # channels, sampling rate
```

```
print(raw.ch_names) # list of signal channels (EEG, ECG, etc.)
```

```
print(raw.times[-1]) # total duration in seconds
```

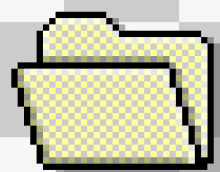
Data Split

60%



Train

20%



VAL

20%



Test



preprocessing

1. Signal Processing Steps

For each subject:

- Select target channel (EOG-R)
- Apply bandpass filtering (0.5–20 Hz)
- Resample to 100 Hz
- Normalize (Z-score normalization)
- Clip extreme values (-5 to 5)

2. Segmentation (Epoching)

- Signal is divided into 30-second epochs
- Each epoch = one training sample

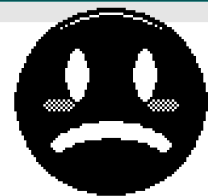
**Save the refined data as
NumPy .npz arrays.**

NPZ File Storage

Processed data is saved in: NumPy format (.npz)

- $X \rightarrow$ input signals (epochs)
- $y \rightarrow$ labels (sleep stages)

Before Processing:



- Raw EDF signal (continuous hours)
- Unstructured biomedical waveform
- No labels aligned per sample

After Processing:

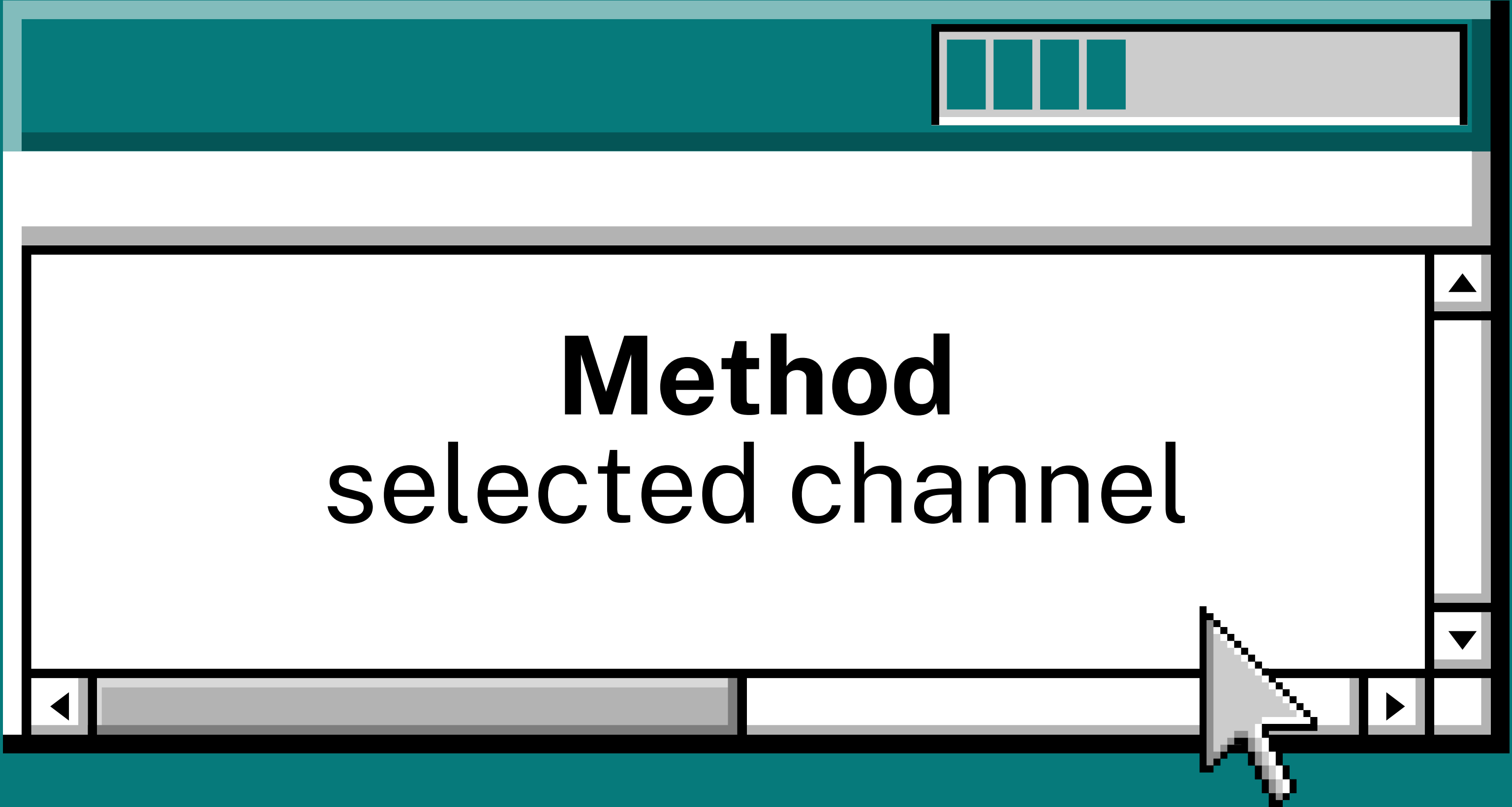


- Fixed-size epochs (30 sec each)
- Normalized signals
- Each segment has a label
- Ready for deep learning

NPZ File Storage

Why NPZ format?

- Faster loading during training
- Reduces memory usage



Method
selected channel

EOG R

Why right ?

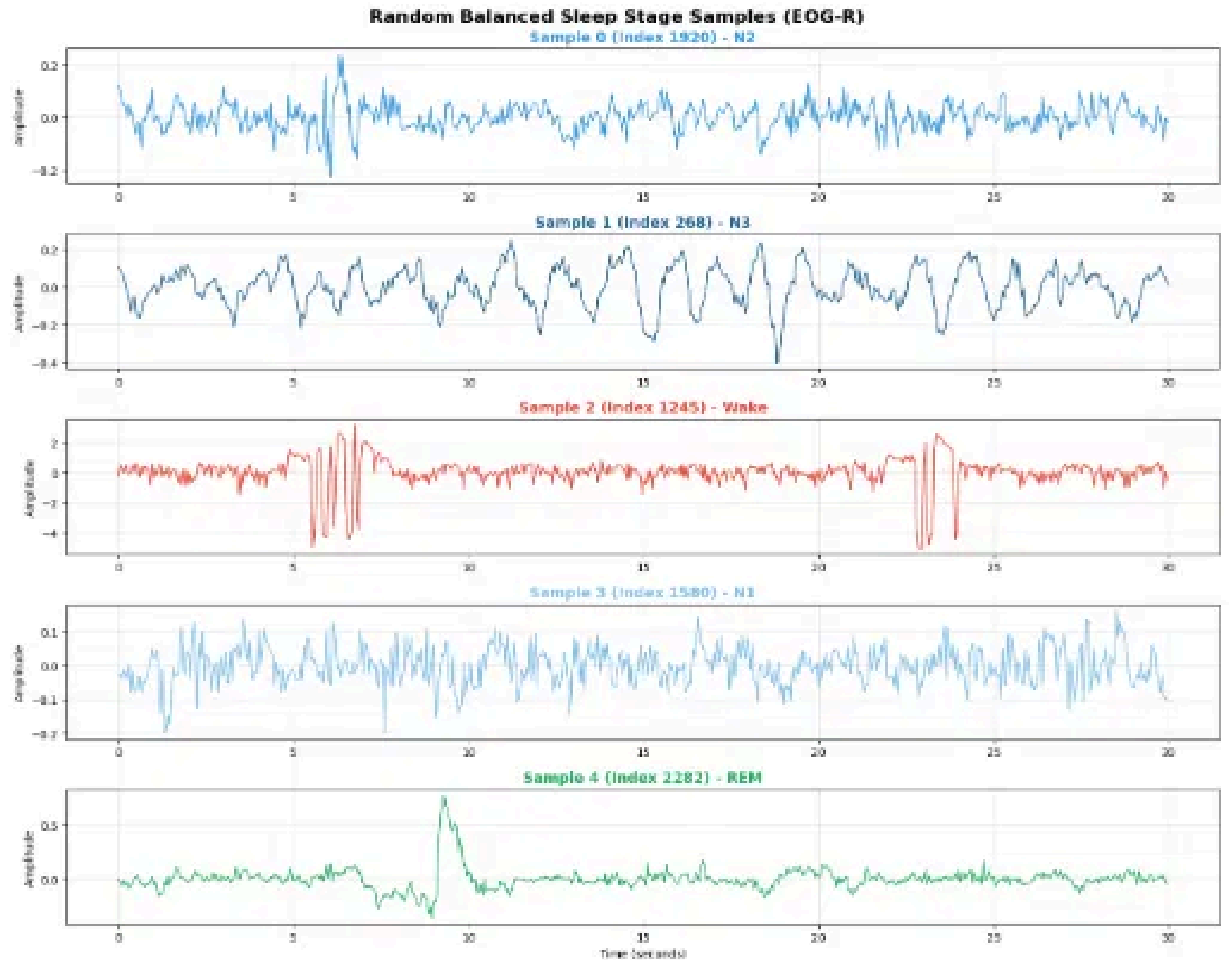
No significant physiological difference is expected between left and right EOG channels for sleep staging.

[Paper link](#)



Preprocessing Data Visualization (EOG-R)

the electrooculography (EOG-R) signal samples after the preprocessing steps, ready for deep learning model consumption. Each segment represents a 30-second epoch, standardized and resampled to 100 Hz, showcasing the distinct patterns characteristic of different sleep stages.



Preprocessing Data Visualization (EOG-R)

Wake (W)

Visual Pattern

High-amplitude, sharp deflections from rapid eye movements and blinks.

N1 (Light Sleep)

Visual Pattern

Low-amplitude, irregular signals as sharp movements disappear.

N2 (Light Sleep)

Visual Pattern

More stable than N1, with specific rhythmic oscillations.

N3 (Deep Sleep)

Visual Pattern

Slow, high-amplitude delta-like waves with minimal eye movement.

REM (Dream Sleep)

Visual Pattern

Low-amplitude background with sudden, sharp "bursts" of eye movement.

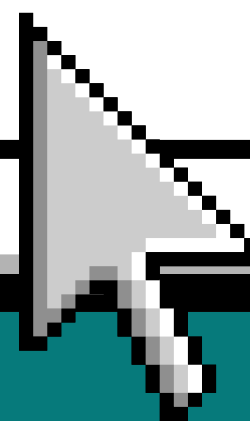




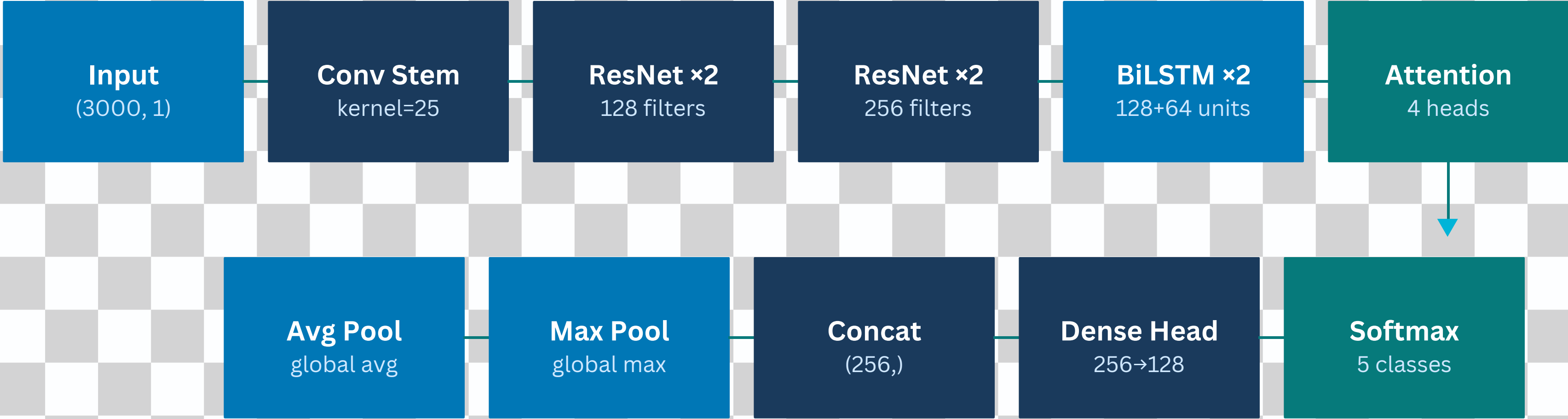
MODEL

CNN + BiLSTM + Temporal Attention

DEEP LEARNING MODEL



Architecture Overview



Shape at each stage:



Output shapes (time steps, channels)

Bidirectional LSTM

→ LSTM Forward (128 units) → reads LEFT to RIGHT → builds context from the PAST

← LSTM Backward (128 units) ← reads RIGHT to LEFT ← builds context from the FUTURE

CONCATENATE → (46, 256)

Why Bidirectional?

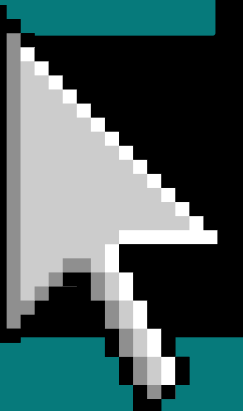
A regular LSTM only sees the past. BiLSTM sees both past AND future for each time step. In sleep staging, what comes after an event matters as much as what came before.

Why two LSTM layers?

First layer captures raw temporal dependencies between time steps. Second layer builds higher-level abstractions on top of those — learning more complex sequential patterns.

recurrent_dropout=0.1

Drops connections INSIDE the LSTM cell — stronger regularization than regular dropout for recurrent layers. Prevents the model from memorizing specific sequences.



Temporal Self-Attention

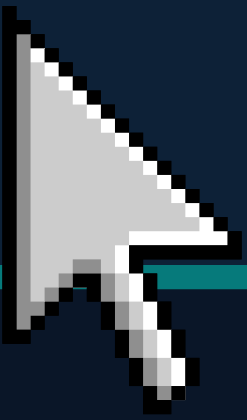
Core Idea: allows the model to look at ANY two time points simultaneously and learn relationships between them – regardless of distance.

Multi-Head Attention	Feed-Forward Network (FFN)	Layer Normalization	Residual Connections
Splits sequence into 4 heads, each learning different relationships. Each head asks: 'How much should time-point A attend to time-point B?'	Two Dense layers applied to each time step independently. Bottleneck (embed×2) forces the model to compress information.	Applied twice – after attention and after FFN. Normalizes activations within each layer for stable training.	Both attention and FFN outputs are added back to their inputs. If the block learns nothing useful, the input passes through unchanged.

A blink at second 5 might relate to an eye movement at second 25. CNNs/LSTMs struggle with very long-range dependencies – attention handles this directly.



Dual Pooling + Classification Head



DUAL POOLING

AVG POOL

GlobalAveragePooling1D
Captures smooth trends and overall behavior of the 30-second epoch. Good for gradual stage transitions.

MAX POOL

GlobalMaxPooling1D
Captures peak activations and strongest features. Good for detecting specific events like REM burst patterns.

CONCATENATE (128,) + (128,) → (256,)

Combining both gives richer information than either alone.

CLASSIFICATION HEAD

Dense(256, GELU)

GELU: smoother than ReLU, works better with attention

BatchNormalization

Normalizes activations between layers for stable training

Dropout(0.4)

Strong regularization — model is large relative to dataset

Dense(128, GELU)

Further compression of learned representations

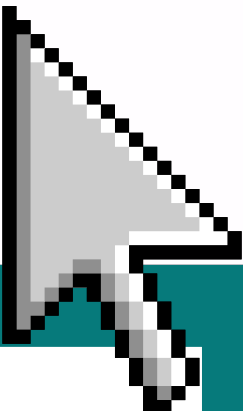
Dropout(0.3)

Second regularization layer

Dense(5, Softmax)

Outputs 5 probabilities that sum to 1.0

Parameter Count & Model Stats



2,411,509

Total Parameters

2,407,029

Trainable

4,480

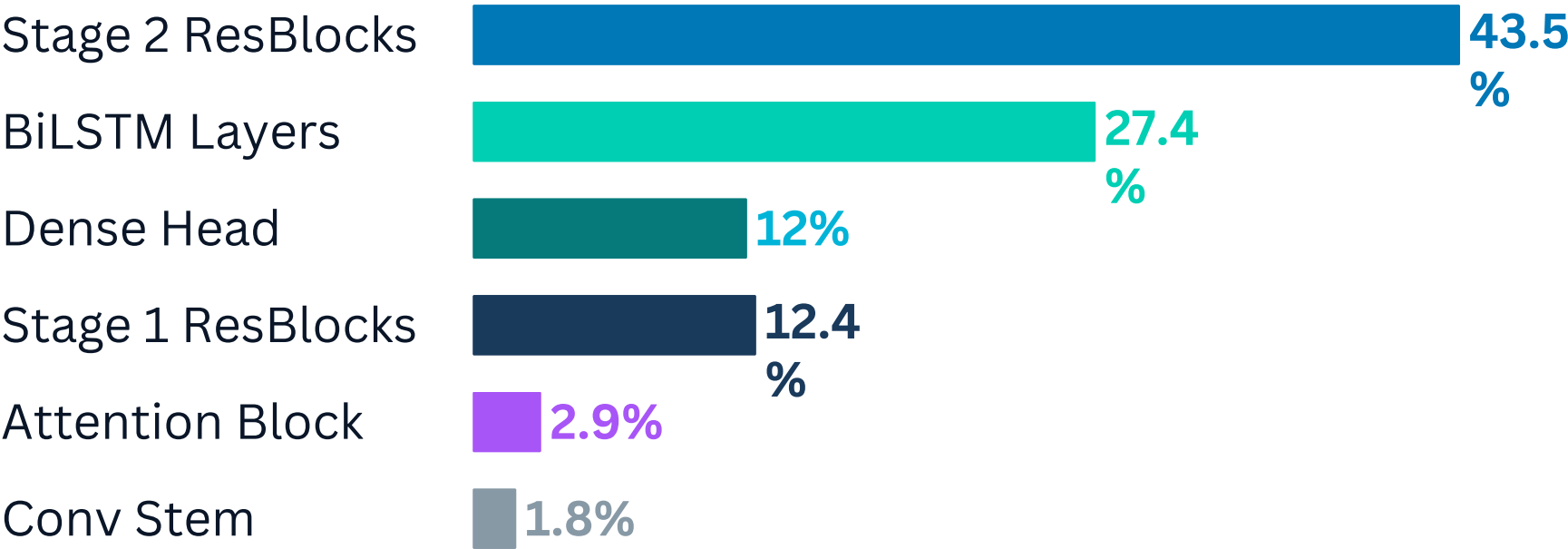
Non-trainable (BN)

9.20 MB

Model Size

Total params: 2,411,509 (9.20 MB)
Trainable params: 2,407,029 (9.18 MB)
Non-trainable params: 4,480 (17.50 KB)
Total params: 2,411,509
Input shape : (None, 3000, 1)
Output shape: (None, 5)

Parameter Distribution by Component



Why this size?

Stage 2 is 43% of params

256 filters × large kernels = most feature extraction power is here.

BiLSTM is 27%

Hidden size 128+64 × 4 (forward/backward × 2 layers) adds up fast.

Is 2.4M too many?

No – it's appropriate for the signal complexity. EEG models often have 10M+.

Non-trainable = 4,480

BatchNorm statistics (mean, variance) – frozen during inference.



Training Strategy

```
def get_callbacks(model_path="best_model_eogr.keras"):
    return [
        callbacks.ReduceLROnPlateau(
            monitor='val_loss', factor=0.2,
            patience=3, min_lr=1e-7, verbose=1
        ),
        callbacks.ModelCheckpoint(
            model_path, monitor='val_accuracy',
            save_best_only=True, verbose=1
        ),
        callbacks.EarlyStopping(
            monitor='val_loss', patience=15,
            restore_best_weights=True, verbose=1
        ),
    ]
```

Callbacks

- **ReduceLROnPlateau**
factor 0.2 · patience 3
- **ModelCheckpoint**
saves best val_accuracy
- **EarlyStopping**
patience 15 · restores best

Epochs

30

Batch size

32

LR

5e-5

Weight decay

1e-4

Optimizer: AdamW · Loss: Sparse Categorical Crossentropy



results

results

Epoch 17/30

280/280  0s 2s/step - accuracy: 0.6971 - loss: 0.7133

Epoch 17: val_accuracy did not improve from 0.85774

The model approached an optimization plateau around epoch 17.

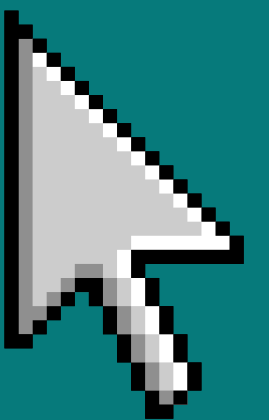
Epoch 26/30

280/280  0s 6s/step - accuracy: 0.7291 - loss: 0.6482

Epoch 26: ReduceLROnPlateau reducing learning rate to 1e-07.

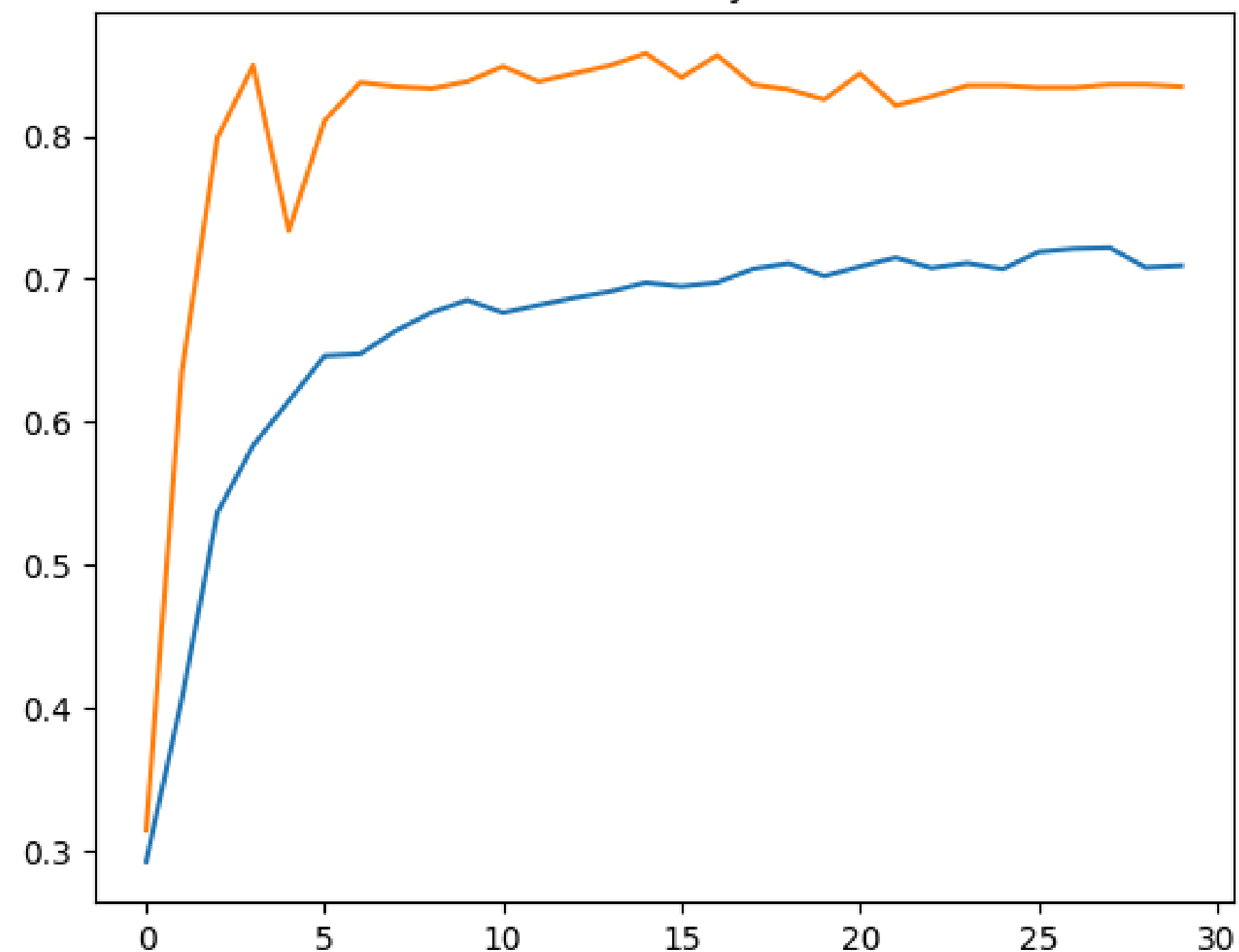
Epoch 26: val_accuracy did not improve from 0.85774

accuracy = 72.9%
Val accuracy = 85.7%

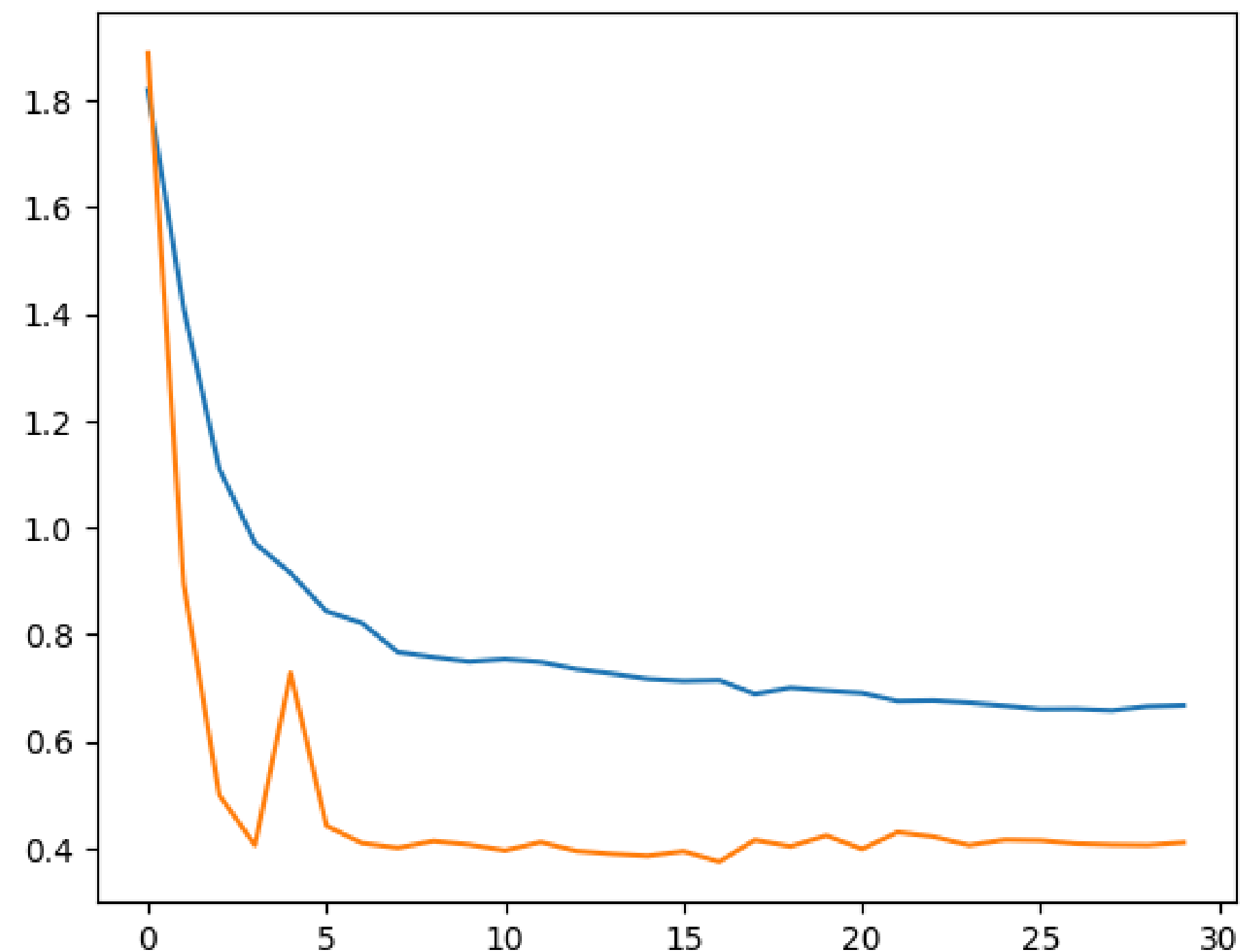


TRAINING HISTORY

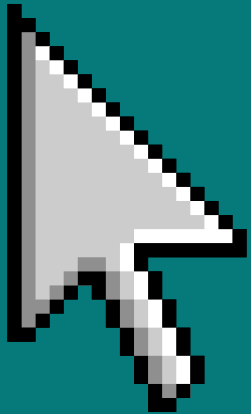
Accuracy



Loss



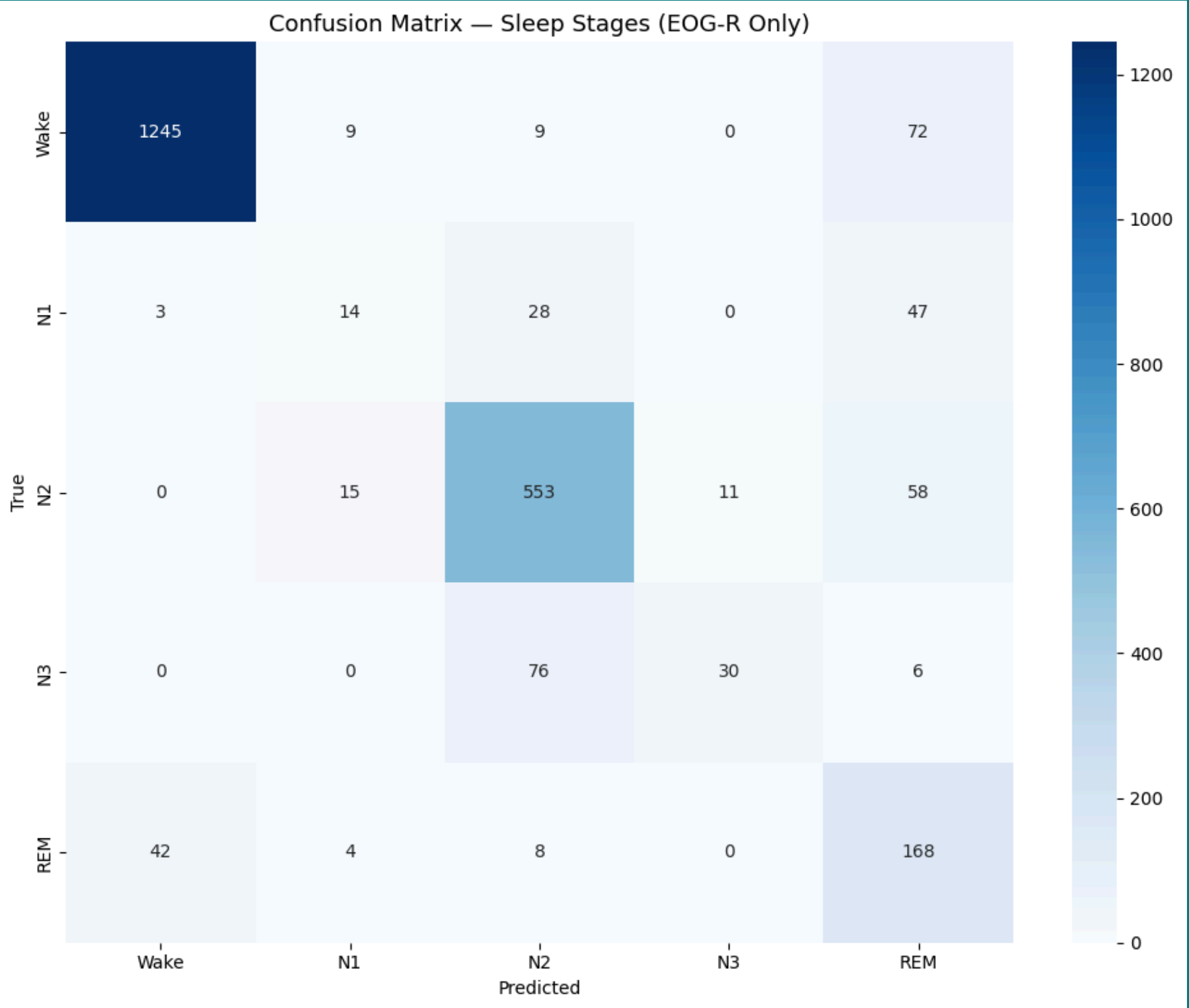
EVALUATION TEST



75/75					52s 638ms/step	
		precision	recall	f1-score	support	
	Wake	0.97	0.93	0.95	1335	
	N1	0.33	0.15	0.21	92	
	N2	0.82	0.87	0.84	637	
	N3	0.73	0.27	0.39	112	
	REM	0.48	0.76	0.59	222	
	accuracy			0.84	2398	
	macro avg	0.67	0.60	0.60	2398	
	weighted avg	0.85	0.84	0.83	2398	

Classification report + Confusion Matrix

Test Set Evaluation				
75/75				
	precision	32s 423ms/step recall	f1-score	support
Wake	0.97	0.93	0.95	1335
N1	0.33	0.15	0.21	92
N2	0.82	0.87	0.84	637
N3	0.73	0.27	0.39	112
REM	0.48	0.76	0.59	222
accuracy			0.84	2398
macro avg	0.67	0.60	0.60	2398
weighted avg	0.85	0.84	0.83	2398

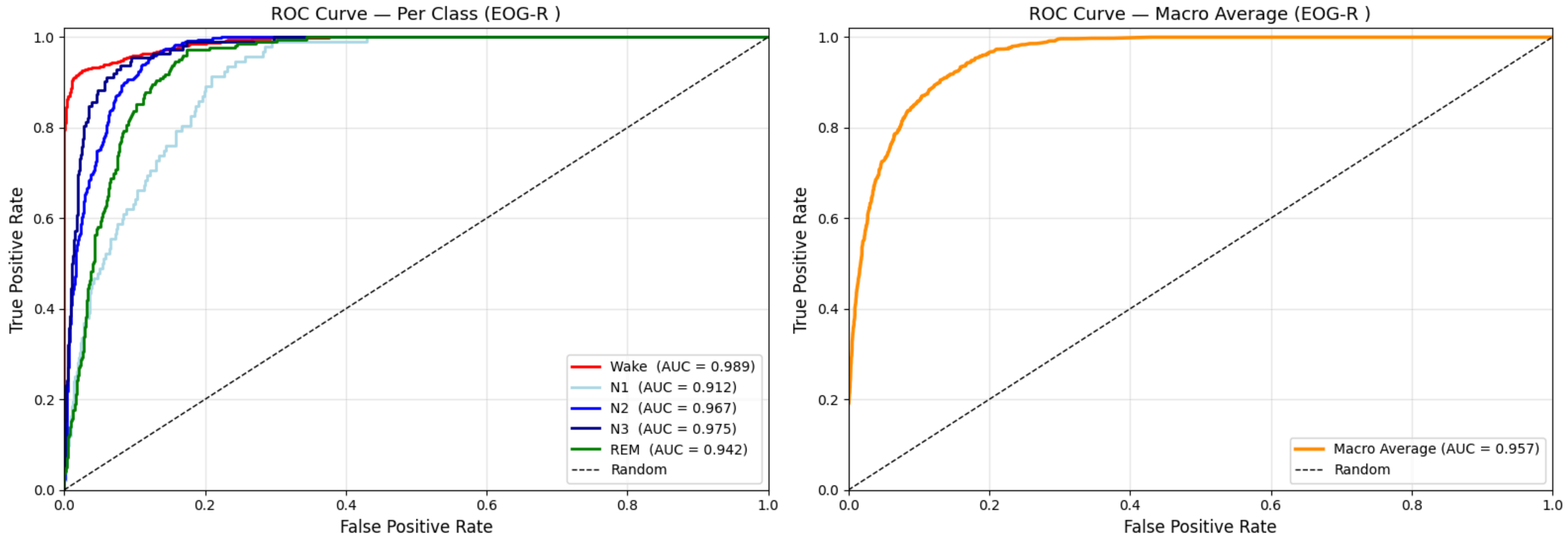


The confusion matrix reveals that most misclassifications occurred between physiologically similar sleep stages, particularly between N2 and N3.

The model is able to distinguish N1 patterns, but the default decision threshold may not be optimal for this minority class.

ROC Curves

ROC Analysis — Sleep Stage Classification



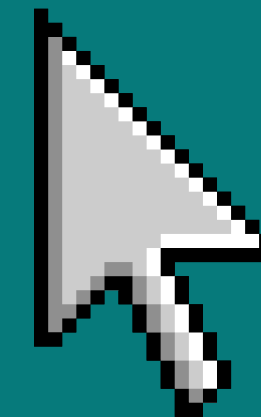
Macro Average: 0.9574



Comparative Performance Analysis

Overall Accuracy	76.30%	84.00%
------------------	--------	--------

[Paper link](#)



Comparative Performance Analysis

precision

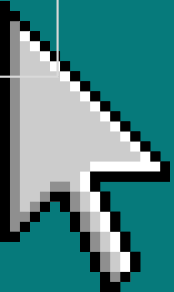
	<i>RESEARCH article</i>	<i>OUR model</i>
Wake Prec.	76.2%	97%
N1 Prec.	33%	33%
N2 Prec.	80.1%	82%
N3 Prec.	73%	73%
REM Prec.	88.0%	48%

recall

	<i>RESEARCH article</i>	<i>OUR model</i>
Wake Rel.	87.9%	93%
N1 Rel.	36.0%	15%
N2 Rel.	83.2%	87%
N3 Rel.	71.2%	27%
REM Rel.	66.4%	76%

f1-score

	<i>RESEARCH article</i>	<i>OUR model</i>
Wake F1	81.60%	95.00%
N1 F1	34.90%	21.00%
N2 F1	81.60%	84.00%
N3 F1	72.50%	39.00%
REM F1	75.70%	59.00%



Thank you

